

背景知识

1.UTXO账户模型

产生背景：

为了解决第一类双花问题（一笔钱花两次）

原理介绍：

我们先来介绍传统的金融模式，你有10元存款，想转给我3元，银行会怎么操作？

很显然，他会将你的账户减3元，将我的账户加3元。

这种交易模式记录的是 交易结果

而UTXO账户模型记录的是 交易过程 下面是简单的例子：

还拿上述例子，你给我转账10元，那么这个机制会做出如下记录：

1. **初始状态**：你的账户有10元，由一个未花费交易输出（UTXO）组成，价值为10元。
2. **转账3元给我**：你想要向我转账3元。这个过程会创建一笔新的交易，包含一个输入和一个输出。
 - 输入：指向之前未花费的10元交易输出，消费掉这个UTXO。
 - 输出：一个新的UTXO，价值为7元，指向我的地址。
3. **交易记录**：区块链账本会记录这笔交易，包含以下信息：
 - 交易ID：由交易内容计算得出的唯一标识符。
 - 输入：指向之前未花费的10元交易输出的引用。
 - 输出：新的7元交易输出，指向我的地址。
 - 其他信息：例如交易的签名等。
4. **更新你的账户余额**：在这笔交易被确认并添加到区块链后，你的账户余额将更新为7元，因为之前的10元交易输出已经被消费掉了。

这样做的好处

1. 结合共识机制，解决了第一类双花问题
2. 使区块链的每一笔交易都可以溯源

如何解决双花问题？

1. 交易机制会先检测账户是否存在这一笔资金
2. 账户会记录下交易源头
3. 通过区块链网络将这一交易广播到每一个账户

在这一机制下，第二笔交易会因账户资金不足而被判定失败

本文只简单介绍一下，至于后续添加节点以及为什么不用时间戳来判断先后的问题以后再说。

2.补档bytes.Buffer

该类型在 Go 标准库中提供了一系列方法，用于对字节序列进行操作。以下是其中一些常用的方法：

1. `Write(p []byte) (n int, err error)`: 将字节序列 `p` 写入到缓冲区。
2. `WriteByte(c byte) error`: 将单个字节 `c` 写入到缓冲区。
3. `WriteRune(r rune) (n int, err error)`: 将 Unicode 字符 `r` 及其 UTF-8 编码写入到缓冲区。
4. `WriteString(s string) (n int, err error)`: 将字符串 `s` 的字节写入到缓冲区。
5. `Read(p []byte) (n int, err error)`: 从缓冲区读取字节到 `p` 中。
6. `ReadByte() (byte, error)`: 从缓冲区读取单个字节。
7. `ReadRune() (r rune, size int, err error)`: 从缓冲区读取一个 UTF-8 编码的 Unicode 字符。
8. `ReadString(delim byte) (string, error)`: 从缓冲区读取直到遇到分隔符 `delim` 的字符串。
9. `Bytes() []byte`: 返回缓冲区的字节切片。
10. `Reset()`: 清空缓冲区，使其长度为 0，但不释放底层的缓冲区。
11. `Len() int`: 返回缓冲区中的字节数。
12. `Cap() int`: 返回缓冲区的容量。

3.补档——range遍历

在Go语言中，`range` 关键字用于迭代数组、切片、字符串、映射等数据结构。

1. 迭代数组和切片

```
package main

import "fmt"

func main() {
    // 迭代数组
    nums := []int{1, 2, 3}
    for _, num := range nums {
        fmt.Println(num)
    }

    // 迭代切片
    fruits := []string{"apple", "banana", "orange"}
    for index, fruit := range fruits {
        fmt.Printf("Index: %d, Fruit: %s\n", index, fruit)
    }
}

//1
//2
//3
//Index: 0, Fruit: apple
//Index: 1, Fruit: banana
//Index: 2, Fruit: orange
```

在这个例子中，`range` 被用于迭代数组和切片。对于数组，`range` 会返回索引和值，而对于切片，则返回索引和对应元素的值。

2. 迭代字符串

```
package main

import "fmt"

func main() {
    // 迭代字符串
    str := "hello"
    for index, char := range str {
        fmt.Printf("Index: %d, Character: %c\n", index, char)
    }
}

//Index: 0, Character: h
//Index: 1, Character: e
//Index: 2, Character: l
//Index: 3, Character: l
//Index: 4, Character: o
```

在这个例子中，`range` 被用于迭代字符串。在Go中，字符串被视为UTF-8编码的字节数组，因此 `range` 会返回每个字符的索引和对应的Unicode码点。

3. 迭代映射 (map)

```
package main

import "fmt"

func main() {
    // 迭代映射
    ages := map[string]int{
        "Alice": 30,
        "Bob":   25,
        "Carol": 35,
    }
    for name, age := range ages {
        fmt.Printf("%s is %d years old\n", name, age)
    }
}

//Alice is 30 years old
//Bob is 25 years old
//Carol is 35 years old
```

在这个例子中，`range` 被用于迭代映射。`range` 会返回映射中的键值对，其中键用于标识映射中的条目，而值则是相应键的值。

源代码

```
package main

import (
    "bytes"
    "crypto/sha256"
    "encoding/binary"
    "encoding/gob"
    "encoding/hex"
    "fmt"
    "log"
    "math"
    "math/big"
    "time"
)

// 常量
const (
    Difficulty = 12
    InitCoin   = 1000
)

// 输出值 货币价值和接收者
type TxOutput struct {
    Value      int
    ToAddress []byte
}

// 输入值 输入所引用的前一笔交易的唯一标识符（哈希） 位置索引 货币来源
type TxInput struct {
    TxID      []byte
    OutIdx     int
    FromAddress []byte
}

// 交易 交易本身的唯一标识符（哈希） 输入值 输出值
type Transaction struct {
    ID        []byte
    Inputs    []TxInput
    Outputs   []TxOutput
}

type Block struct {
    Timestamp    int64
    Hash         []byte
    PrevHash     []byte
    Target       []byte
    Nonce        int64
    Transactions []*Transaction
}
```

```

type Blockchain struct {
    Blocks []*Block
}

// 下面两个函数实现 将整数转化为16进制 并且最后返回字节切片
func Handle(err error) {
    if err != nil {
        log.Panic(err)
    }
}

func ToHexInt(num int64) []byte {
    buff := new(bytes.Buffer)
    err := binary.Write(buff, binary.BigEndian, num)
    Handle(err)
    return buff.Bytes()
}

// 下面两个方法 实现了交易ID的创建
func (tx *Transaction) TxHash() []byte {
    var encoded bytes.Buffer
    var hash [32]byte

    encoder := gob.NewEncoder(&encoded)
    err := encoder.Encode(tx)
    Handle(err)

    hash = sha256.Sum256(encoded.Bytes())
    return hash[:]
}

func (tx *Transaction) SetID() {
    tx.ID = tx.TxHash()
}

// 创建每个区块链的第一个交易——基本交易
func BaseTx(toaddress []byte) *Transaction {
    txIn := TxInput{[]byte{}, -1, []byte{}}
    txOut := TxOutput{InitCoin, toaddress}
    tx := Transaction{[]byte("This is the Base Transaction!"), []TxInput{txIn},
    []TxOutput{txOut}}
    return &tx
}

// 检查输入的地址是否与 TxInput 结构体中的 FromAddress 字段相匹配
func (in *TxInput) FromAddressRight(address []byte) bool {
    return bytes.Equal(in.FromAddress, address)
}

// 检查输出的地址是否与 TxOutput 结构体中的 ToAddress 字段相匹配
func (out *TxOutput) ToAddressRight(address []byte) bool {
    return bytes.Equal(out.ToAddress, address)
}

func CreateBlock(prevhash []byte, txs []*Transaction) *Block {
    block := Block{time.Now().Unix(), []byte{}, prevhash, []byte{}, 0, txs}
    block.Target = block.GetTarget()
}

```

```

    block.Nonce = block.FindNonce()
    block.SetHash()
    return &block
}

func GenesisBlock() *Block {
    tx := BaseTx([]byte("Leo Cao"))
    return CreateBlock([]byte{}, []*Transaction{tx})
}

// 生成一个包含区块中所有交易的唯一标识符 ID 的摘要
func (b *Block) BackTrasactionSummary() []byte {
    txIDs := make([][]byte, 0)
    for _, tx := range b.Transactions {
        txIDs = append(txIDs, tx.ID)
    }
    summary := bytes.Join(txIDs, []byte{})
    return summary
}

func (b *Block) SetHash() {
    information := bytes.Join([][]byte{ToHexInt(b.Timestamp), b.PrevHash,
    b.Target, ToHexInt(b.Nonce), b.BackTrasactionSummary()}, []byte{})
    hash := sha256.Sum256(information)
    b.Hash = hash[:]
}

func (b *Block) GetTarget() []byte {
    target := big.NewInt(1)
    target.Lsh(target, uint(256-Difficulty))
    return target.Bytes()
}

func (b *Block) GetBase4Nonce(nonce int64) []byte {
    data := bytes.Join([][]byte{
        ToHexInt(b.Timestamp),
        b.PrevHash,
        ToHexInt(int64(nonce)),
        b.Target,
        b.BackTrasactionSummary(),
    }, []byte{})
    return data
}

func (b *Block) FindNonce() int64 {
    var intHash big.Int
    var intTarget big.Int
    var hash [32]byte
    var nonce int64
    nonce = 0
    intTarget.SetBytes(b.Target)

    for nonce < math.MaxInt64 {
        data := b.GetBase4Nonce(nonce)
        hash = sha256.Sum256(data)
        intHash.SetBytes(hash[:])
    }
}

```

```

        if intHash.Cmp(&intTarget) == -1 {
            break
        } else {
            nonce++
        }
    }
    return nonce
}

func (b *Block) ValidatePow() bool {
    var intHash big.Int
    var intTarget big.Int
    var hash [32]byte
    intTarget.SetBytes(b.Target)
    data := b.GetBase4Nonce(b.Nonce)
    hash = sha256.Sum256(data)
    intHash.SetBytes(hash[:])
    if intHash.Cmp(&intTarget) == -1 {
        return true
    }
    return false
}

func (bc *BlockChain) AddBlock(txs []*Transaction) {
    newBlock := CreateBlock(bc.Blocks[len(bc.Blocks)-1].Hash, txs)
    bc.Blocks = append(bc.Blocks, newBlock)
}

func CreateBlockChain() *BlockChain {
    blockchain := BlockChain{}
    blockchain.Blocks = append(blockchain.Blocks, GenesisBlock())
    return &blockchain
}

// 判断是不是基本交易
func (tx *Transaction) IsBase() bool {
    return len(tx.Inputs) == 1 && tx.Inputs[0].OutIdx == -1
}

// 查找 未花费交易
func (bc *BlockChain) FindUnspentTransactions(address []byte) []Transaction {
    var unSpentTxs []Transaction
    spentTxs := make(map[string][]int)

    for idx := len(bc.Blocks) - 1; idx >= 0; idx-- {
        block := bc.Blocks[idx]
        for _, tx := range block.Transactions {
            txID := hex.EncodeToString(tx.ID)

            IterOutputs:
            for outIdx, out := range tx.Outputs {
                if spentTxs[txID] != nil {
                    for _, spentOut := range spentTxs[txID] {
                        if spentOut == outIdx {
                            continue IterOutputs
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
}

if out.ToAddressRight(address) {
    unSpentTxs = append(unSpentTxs, *tx)
}

}

if !tx.IsBase() {
    for _, in := range tx.Inputs {
        if in.FromAddressRight(address) {
            inTxID := hex.EncodeToString(in.TxID)
            spentTxs[inTxID] = append(spentTxs[inTxID], in.OutIdx)
        }
    }
}

}

}

return unSpentTxs
}

```

// 计算UTXO 资金和输出索引

```

func (bc *BlockChain) FindUTXOs(address []byte) (int, map[string]int) {
    unspentOuts := make(map[string]int)
    unspentTxs := bc.FindUnspentTransactions(address)
    accumulated := 0

```

work:

```

    for _, tx := range unspentTxs {
        txID := hex.EncodeToString(tx.ID)
        for outIdx, out := range tx.Outputs {
            if out.ToAddressRight(address) {
                accumulated += out.Value
                unspentOuts[txID] = outIdx
                continue work
            }
        }
    }

    return accumulated, unspentOuts
}

```

```

func (bc *BlockChain) FindSpendableOutputs(address []byte, amount int) (int,
map[string]int) {
    unspentOuts := make(map[string]int)
    unspentTxs := bc.FindUnspentTransactions(address)
    accumulated := 0

```

work:

```

    for _, tx := range unspentTxs {
        txID := hex.EncodeToString(tx.ID)
        for outIdx, out := range tx.Outputs {
            if out.ToAddressRight(address) && accumulated < amount {
                accumulated += out.Value
                unspentOuts[txID] = outIdx
            }
        }
    }

```



```

        if accumulated >= amount {
            break work
        }
        continue work
    }
}
}
return accumulated, unspentOuts
}

func (bc *Blockchain) CreateTransaction(from, to []byte, amount int)
(*Transaction, bool) {
    var inputs []TxInput
    var outputs []TxOutput

    acc, validOutputs := bc.FindSpendableOutputs(from, amount)
    if acc < amount {
        fmt.Println("Not enough coins!")
        return &Transaction{}, false
    }

    for txid, outidx := range validOutputs {
        txID, err := hex.DecodeString(txid)
        Handle(err)
        input := TxInput{txID, outidx, from}
        inputs = append(inputs, input)
    }

    outputs = append(outputs, TxOutput{amount, to})
    if acc > amount {
        outputs = append(outputs, TxOutput{acc - amount, from})
    }

    tx := Transaction{nil, inputs, outputs}
    tx.SetID()

    return &tx, true
}

func (bc *Blockchain) Mine(txs []*Transaction) {
    bc.AddBlock(txs)
}

func main() {
    txPool := make([]*Transaction, 0)
    var tempTx *Transaction
    var ok bool
    var property int

    chain := CreateBlockchain()
    property, _ = chain.FindUTXOs([]byte("Always"))
    fmt.Println("Balance of Always: ", property)

    tempTx, ok = chain.CreateTransaction([]byte("Always"), []byte("ManBa"), 50)
    if ok {
        txPool = append(txPool, tempTx)
    }
}

```

```

}
chain.Mine(txPool)
txPool = make([]*Transaction, 0)
property, _ = chain.FindUTXOs([]byte("Always"))
fmt.Println("Balance of Always: ", property)

tempTx, ok = chain.CreateTransaction([]byte("ManBa"), []byte("KFC"), 100)
if ok {
    txPool = append(txPool, tempTx)
}

tempTx, ok = chain.CreateTransaction([]byte("Always"), []byte("ManBa"), 200)
if ok {
    txPool = append(txPool, tempTx)
}

tempTx, ok = chain.CreateTransaction([]byte("ManBa"), []byte("KFC"), 100)
if ok {
    txPool = append(txPool, tempTx)
}
chain.Mine(txPool)
txPool = make([]*Transaction, 0)

property, _ = chain.FindUTXOs([]byte("Always"))
fmt.Println("Balance of Always: ", property)
property, _ = chain.FindUTXOs([]byte("ManBa"))
fmt.Println("Balance of ManBa: ", property)
property, _ = chain.FindUTXOs([]byte("KFC"))
fmt.Println("Balance of KFC: ", property)

for _, block := range chain.Blocks {
    fmt.Printf("Timestamp: %d\n", block.Timestamp)
    fmt.Printf("hash: %x\n", block.Hash)
    fmt.Printf("Previous hash: %x\n", block.PrevHash)
    fmt.Printf("nonce: %d\n", block.Nonce)
    fmt.Println("Proof of work validation:", block.ValidatePow())
}

tempTx, ok = chain.CreateTransaction([]byte("ManBa"), []byte("Always"), 300)
if ok {
    txPool = append(txPool, tempTx)
}

tempTx, ok = chain.CreateTransaction([]byte("KFC"), []byte("Always"), 100)
if ok {
    txPool = append(txPool, tempTx)
}

chain.Mine(txPool)
txPool = make([]*Transaction, 0)

for _, block := range chain.Blocks {
    fmt.Printf("Timestamp: %d\n", block.Timestamp)
    fmt.Printf("hash: %x\n", block.Hash)
    fmt.Printf("Previous hash: %x\n", block.PrevHash)
    fmt.Printf("nonce: %d\n", block.Nonce)

```

```

        fmt.Println("Proof of work validation:", block.ValidatePow())
    }

    property, _ = chain.FindUTXOs([]byte("Always"))
    fmt.Println("Balance of Always: ", property)
    property, _ = chain.FindUTXOs([]byte("ManBa"))
    fmt.Println("Balance of ManBa: ", property)
    property, _ = chain.FindUTXOs([]byte("KFC"))
    fmt.Println("Balance of KFC: ", property)
}

```

运行结果:

```

Balance of Always: 648
Balance of Always: 598
Not enough coins!
Not enough coins!
Balance of Always: 398
Balance of ManBa: 250
Balance of KFC: 0
Timestamp: 1712235727
hash: e2f4a7591117cedbd96027dee26ba8405610160ff3806841233a7c329f1ffc36
Previous hash:
nonce: 12455
Proof of work validation: true
Timestamp: 1712235727
hash: 37990380ee6b0063531a00fed14e0dfbb945dd964e368b45b710fb9426745201
Previous hash: e2f4a7591117cedbd96027dee26ba8405610160ff3806841233a7c329f1ffc36
nonce: 1662
Proof of work validation: true
Timestamp: 1712235727
hash: 14b89d872c2a2177f10e6fff7755f25aeb4ff57e3256c90f9b4430f81bf575c9
Previous hash: 37990380ee6b0063531a00fed14e0dfbb945dd964e368b45b710fb9426745201
nonce: 1118
Proof of work validation: true
Not enough coins!
Not enough coins!
Timestamp: 1712235727
hash: e2f4a7591117cedbd96027dee26ba8405610160ff3806841233a7c329f1ffc36
Previous hash:
nonce: 12455
Proof of work validation: true
Timestamp: 1712235727
hash: 37990380ee6b0063531a00fed14e0dfbb945dd964e368b45b710fb9426745201
Previous hash: e2f4a7591117cedbd96027dee26ba8405610160ff3806841233a7c329f1ffc36
nonce: 1662
Proof of work validation: true
Timestamp: 1712235727
hash: 14b89d872c2a2177f10e6fff7755f25aeb4ff57e3256c90f9b4430f81bf575c9
Previous hash: 37990380ee6b0063531a00fed14e0dfbb945dd964e368b45b710fb9426745201
nonce: 1118
Proof of work validation: true
Timestamp: 1712235727
hash: 549dd7ae31c3a54a69a6157b54e49520484c8d2abb6468d0abe07649c3d868bf

```

```
Previous hash: 14b89d872c2a2177f10e6fff7755f25aeb4ff57e3256c90f9b4430f81bf575c9
nonce: 7739
Proof of work validation: true
Balance of Always: 398
Balance of ManBa: 250
Balance of KFC: 0
```

按照工作分成几包

utils包

该包用于存储一些 类型转化 错误判断等和账本无直接关系的工具函数

util.go

```
package utils

import (
    "bytes"
    "encoding/binary"
    "log"
)

//该包用于存储一些 类型转化 错误判断等和账本无直接关系但是要用到的函数

// 用于处理错误的通用函数 Handle(err error)
// 作用是在出现错误时，记录错误信息并终止程序的执行
func Handle(err error) {
    if err != nil {
        //在Go语言中，nil是一个预定义的常量，用于表示指针、接口、切片、map、函数和通道的零值或空值。
        //它是一种特殊的零值，表示该类型的变量不指向任何有效的内存地址或不包含任何有效的数值。
        log.Panic(err)
    }
    //如果 err 不为 nil，意味着有错误发生
}

// 函数 将整数转化为16进制 并且最后返回字节切片
func ToHexInt(num int64) []byte {
    buff := new(bytes.Buffer)
    err := binary.Write(buff, binary.BigEndian, num)
    //如果在写入过程中出现任何问题，会将错误存储在变量 err 中
    Handle(err) //相当于这两行实现了判断写入缓存区是否报错
    return buff.Bytes() //按字节返回
}
```

constcoe包

存储常量

constcoe.go

```
package constcoe

const (
    Difficulty = 12
    InitCoin   = 648 //区块链在创建时的总的比特币数目
    // 初始交易信息因为是凭空产生的，其Input指向一个为空的交易信息中的序号为-1的Output
)
```

transaction包

transaction.go

```
package transaction

import (
    "GOBLOCKCHAIN/constcoe"
    "GOBLOCKCHAIN/utils"
    "bytes"
    "crypto/sha256"
    "encoding/gob"
)

// 下列结构用于记录一笔交易
type Transaction struct {
    ID        []byte    //自身的ID值(哈希值) 由一组TxInput与一组TxOutput构成
    Inputs    []TxInput //TxInput用于标记支持我们本次转账的前置的交易信息的TxOutput
    Outputs   []TxOutput //而TxOutput记录我们本次转账的amount和Receiver
}

//下面两个函数 实现计算每个transaction的哈希值的功能

// TxHash返回交易信息的哈希值
func (tx *Transaction) TxHash() []byte {
    var encoded bytes.Buffer
    var hash [32]byte

    encoder := gob.NewEncoder(&encoded)
    //gob, 其功能主要是序列化结构体, 与json有些像但是更方便
    err := encoder.Encode(tx)
    utils.Handle(err)

    hash = sha256.Sum256(encoded.Bytes())
    return hash[:]
}

// SetID设置每个交易信息的ID值, 也就是哈希值
func (tx *Transaction) SetID() {
```

```

    tx.ID = tx.TxHash() //调用 TxHash() 方法计算的哈希值设置为交易 (tx) 的 ID 字段
}

// 创建一个基本的交易 (transaction)，并返回该交易的指针
func BaseTx(toaddress []byte) *Transaction {
    txIn := TxInput{[]byte{}, -1, []byte{}}
    //txIn: 一个空的交易输入 (TxInput)，表示没有引用任何之前的交易输出 (UTXO) 作为输入
    txOut := TxOutput{constcoe.InitCoin, toaddress}
    //txOut: 一个交易输出 (TxOutput)，包含一个常量 InitCoin 作为金额，以及传入函数的
    toaddress 作为收款地址。
    tx := Transaction{[]byte("This is the Base Transaction!"), []TxInput{txIn},
    []TxOutput{txOut}}
    //tx: 一个交易 (Transaction) 结构，包含了一些元数据 (如交易的描述)，以及上述创建的交易输入和输出。
    return &tx
}

// 用是判断一个交易是否是基础交易
func (tx *Transaction) IsBase() bool {
    return len(tx.Inputs) == 1 && tx.Inputs[0].OutIdx == -1
    //检查交易的输入是否只有一个，并且该输入的输出索引 (OutIdx) 是否为 -1
    //如果是，那么这个交易被认为是基础交易
}

```

inoutput.go

位置索引是什么？

在区块链交易中，位置索引 (OutIdx) 表示特定输出在交易输出列表中的位置或索引。每个输出都有一个唯一的位置索引，从0开始递增，用于标识输出在交易中的位置。

举个例子，假设有一个包含三个输出的交易，它们分别是：

1. 输出0: 10 BTC 发往地址A
2. 输出1: 5 BTC 发往地址B
3. 输出2: 3 BTC 发往地址C

在这个例子中，输出0的位置索引为0，输出1的位置索引为1，输出2的位置索引为2。当需要引用或使用这些输出时，可以通过它们的位置索引来确定具体是哪个输出。

TxID是什么？ID又是什么？

1. **TxID (TxInput结构体中的字段)**：这个字段表示输入所引用的前一笔交易的唯一标识符。
2. 在区块链中，一个交易的输入通常会引用之前某个交易的输出，以便使用这些输出的金额来创建新的交易。因此，TxID 表示被引用的交易的唯一标识符。
3. **ID (Transaction结构体中的字段)**：这个字段表示交易本身的唯一标识符。
4. 交易的 ID 通过对整个交易内容进行哈希计算而生成，用于唯一标识一笔交易。在区块链中，ID 被用于区分不同的交易，而且也可以被用来验证交易的完整性。

代码

```
package transaction

import "bytes"

type TxOutput struct { //交易输出
    Value      int      // 表示输出的货币数量或价值 整形
    ToAddress []byte //表示交易输出的接收方地址 字节数组 用于表示钱包地址或者公钥哈希等
}

type TxInput struct { //交易输入
    TxID []byte // 表示该输入所引用的先前交易的唯一标识符 字节数组 用于指定该输入的来源
    OutIdx int    //表示在引用的先前交易中输出的索引 用于指定该输入来自该交易中的哪个输出
    //OutIdx是具体指明是前置交易信息中的第几个Output
    FromAddress []byte //表示该输入的发送方地址 字节数组 用于表示钱包地址或者公钥哈希等
}

// 验证交易输入的来源地址是否与指定的地址匹配
func (in *TxInput) FromAddressRight(address []byte) bool {
    return bytes.Equal(in.FromAddress, address)
    //比较 TxInput 结构体中的 FromAddress 字段和传入的 address 参数是否相等，如果相等则返回 true，否则返回 false
}

// 检查一个交易是否为基础交易
func (out *TxOutput) ToAddressRight(address []byte) bool {
    return bytes.Equal(out.ToAddress, address)
}

//在比特币和许多其他加密货币的交易模型中，基础交易是一种特殊类型的交易，它没有输入，只有一个输出。
//基础交易通常用于创建新的货币（coinbase）或者奖励矿工。
//因此，这个函数检查一个交易是否只有一个输入且该输入的输出索引为-1，如果是，则返回true，否则返回false。
```

blockchain包

block.go

```
package blockchain

import (
    "GOBLOCKCHAIN/transaction"
    "GOBLOCKCHAIN/utils"
    "bytes"
    "crypto/sha256"
    "time"
)

type Block struct {
    Timestamp int64
    Hash      []byte
}
```

```

    PrevHash    []byte
    Target      []byte
    Nonce       int64
    Transactions []*transaction.Transaction //用于存储一个或多个交易的信息
}

// 创建节点
func CreateBlock(prevhash []byte, txs []*transaction.Transaction) *Block {
    block := Block{time.Now().Unix(), []byte{}, prevhash, []byte{}, 0, txs}
    //这个区块存储的一个要素解释一条交易信息
    block.Target = block.GetTarget()
    block.Nonce = block.FindNonce()
    block.SetHash()
    return &block
}

// 创建创世节点
func GenesisBlock() *Block {
    tx := transaction.BaseTx([]byte("Always"))
    return CreateBlock([]byte{}, []*transaction.Transaction{tx})
}

func (b *Block) BackTrasactionSummary() []byte {
    txIDs := make([][]byte, 0)
    //二维切片 每一行存储一条交易信息
    for _, tx := range b.Transactions {
        txIDs = append(txIDs, tx.ID)
    } //将每个交易的 ID 添加到 txIDs 这个二维切片中的新行中
    summary := bytes.Join(txIDs, []byte{})
    //相当于将这个二维切片连成一个一维切片，每行之间用[]byte{}隔开
    return summary
}

// 合信息 算哈希
func (b *Block) SetHash() {
    information := bytes.Join([][]byte{utils.ToHexInt(b.Timestamp), b.PrevHash,
    b.Target, utils.ToHexInt(b.Nonce), b.BackTrasactionSummary()}, []byte{})
    hash := sha256.Sum256(information)
    b.Hash = hash[:]
}

```

proofofwork.go

```

package blockchain

import (
    "GOBLOCKCHAIN/constcoe"
    "GOBLOCKCHAIN/utils"
    "bytes"
    "crypto/sha256"
    "math"
    "math/big"
)

```



```
// 基本不变 相当于将输入的字符串换成交易信息
func (b *Block) GetTarget() []byte {
    target := big.NewInt(1)
    target.Lsh(target, uint(256-constcoe.Difficulty))
    return target.Bytes()
}

func (b *Block) GetBase4Nonce(nonce int64) []byte {
    data := bytes.Join([][]byte{
        utils.ToHexInt(b.Timestamp),
        b.PrevHash,
        utils.ToHexInt(int64(nonce)),
        b.Target,
        b.BackTrasactionSummary(),
    },
        []byte{},
    )
    return data
}

func (b *Block) FindNonce() int64 {
    var intHash big.Int
    var intTarget big.Int
    var hash [32]byte
    var nonce int64
    nonce = 0
    intTarget.SetBytes(b.Target)

    for nonce < math.MaxInt64 {
        data := b.GetBase4Nonce(nonce)
        hash = sha256.Sum256(data)
        intHash.SetBytes(hash[:])
        if intHash.Cmp(&intTarget) == -1 {
            break
        } else {
            nonce++
        }
    }
    return nonce
}

func (b *Block) validatePow() bool {
    var intHash big.Int
    var intTarget big.Int
    var hash [32]byte
    intTarget.SetBytes(b.Target)
    data := b.GetBase4Nonce(b.Nonce)
    hash = sha256.Sum256(data)
    intHash.SetBytes(hash[:])
    if intHash.Cmp(&intTarget) == -1 {
        return true
    }
    return false
}
```

blockchain.go

```
package blockchain

import (
    "GOBLOCKCHAIN/transaction"
    "GOBLOCKCHAIN/utils"
    "encoding/hex"
    "fmt"
)

type Blockchain struct {
    Blocks []*Block //blocks是该结构体中的一个 区块类型 的成员变量
}

// 下面两个函数 就是添加进链的信息换成了交易内容 其余不变
func (bc *Blockchain) AddBlock(txs []*transaction.Transaction) {
    newBlock := CreateBlock(bc.Blocks[len(bc.Blocks)-1].Hash, txs)
    bc.Blocks = append(bc.Blocks, newBlock)
}

func CreateBlockchain() *Blockchain {
    blockchain := Blockchain{}
    blockchain.Blocks = append(blockchain.Blocks, GenesisBlock())
    return &blockchain
}

// 用于查找指定地址 (address) 的未花费交易 (unspent transactions)
// 该函数会遍历区块链中的每一个区块，然后对每个区块中的交易进行处理，找出未被花费的交易输出 UTXO
func (bc *Blockchain) FindUnspentTransactions(address []byte)
[]transaction.Transaction {
    var unSpentTxs []transaction.Transaction
    //初始化一个空的未花费交易数组 unSpentTxs，用于存储找到的未花费交易
    spentTxs := make(map[string][]int)
    //初始化一个 spentTxs 映射，用于跟踪已经被花费的交易输出
    //该映射的键是交易 ID 的十六进制表示，值是一个整数切片，表示对应交易中已经被花费的输出索引

    //外部循环
    for idx := len(bc.Blocks) - 1; idx >= 0; idx-- {
        //从最新的区块开始向前遍历区块链中的每一个区块
        block := bc.Blocks[idx] //现在，我们进入了一个区块中
        //对于每个区块，遍历其中的每一笔交易
        for _, tx := range block.Transactions {
            txID := hex.EncodeToString(tx.ID) //将交易ID转换为十六进制字符串格式

            IterOutputs: //判断是不是第一个交易
            for outIdx, out := range tx.Outputs {
                if spentTxs[txID] != nil {
                    //检查交易ID没有在已花费列表中 下列循环
                    for _, spentOut := range spentTxs[txID] {
                        if spentOut == outIdx {
                            continue IterOutputs //如果发现已经存在，就不执行
                        }
                    }
                }
            }
            //ToAddress判断
        }
    }
}
```

```

    }
    }
}

if out.ToAddressRight(address) { //判断是不是第一个交易
    unSpentTxs = append(unSpentTxs, *tx)
    //检查输出是否属于指定地址，如果是，则将该交易添加到未花费交易列表中。
}
} //简单来说 就是找到属于某个用户的交易

//如果不是第一个交易 进行判断
if !tx.IsBase() {
    for _, in := range tx.Inputs {
        if in.FromAddressRight(address) {
            inTxID := hex.EncodeToString(in.TxID)
            spentTxs[inTxID] = append(spentTxs[inTxID], in.OutIdx)
            //检查输入是否来自指定地址，如果是，则将该输入标记为已花费
        }
    }
}
}
}
//返回找到的未花费交易数组
return unSpentTxs
}

```

// 用于查找指定地址（address）的未花费交易输出（Unspent Transaction Outputs, UTXOs）的总金额和对应的输出索引

```

func (bc *Blockchain) FindUTXOs(address []byte) (int, map[string]int) {
    // 创建一个 map 用于存储未花费的输出
    unspentOuts := make(map[string]int)
    // 查找包含未花费输出的交易
    unspentTxs := bc.FindUnspentTransactions(address)
    // 初始化累计变量
    accumulated := 0

```

work:

```

    // 遍历未花费交易
    for _, tx := range unspentTxs {
        // 将交易ID转换为字符串形式
        txID := hex.EncodeToString(tx.ID)
        // 遍历交易的输出
        for outIdx, out := range tx.Outputs {
            // 检查输出是否属于指定地址
            if out.ToAddressRight(address) {
                // 如果是，则累加输出的价值到累计变量中
                accumulated += out.Value
                // 将交易ID和输出索引添加到未花费输出的 map 中
                unspentOuts[txID] = outIdx
                // 继续到标签为 work 的循环，因为一笔交易只能有一个输出与地址相关联
                continue work
            }
        }
    }
}
// 返回累计的价值和未花费输出的 map
return accumulated, unspentOuts

```

```

}

// 用于查找指定地址（address）可以用来支付指定金额（amount）的未花费交易输出（Unspent Transaction Outputs, UTXOs）的总金额和对应的输出索引。
func (bc *Blockchain) FindSpendableOutputs(address []byte, amount int) (int, map[string]int) {
    // 创建一个 map 用于存储未花费的输出
    unspentOuts := make(map[string]int)
    // 查找包含未花费输出的交易
    unspentTxns := bc.FindUnspentTransactions(address)
    // 初始化累计变量，用于跟踪累计的金额
    accumulated := 0

```

work:

```

// 遍历未花费交易
for _, tx := range unspentTxns {
    // 将交易ID转换为字符串形式
    txID := hex.EncodeToString(tx.ID)
    // 遍历交易的输出
    for outIdx, out := range tx.Outputs {
        // 检查输出是否属于指定地址且累计的金额小于支付金额
        if out.ToAddressRight(address) && accumulated < amount {
            // 累加输出的价值到累计变量中
            accumulated += out.Value
            // 将交易ID和输出索引添加到未花费输出的 map 中
            unspentOuts[txID] = outIdx
            // 如果累计的金额达到或超过支付金额，则跳出循环
            if accumulated >= amount {
                break work
            }
            // 继续到标签为 work 的循环，因为一笔交易只能有一个输出与地址相关联
            continue work
        }
    }
}

// 返回累计的价值和未花费输出的 map
return accumulated, unspentOuts
}

```

// 该函数接收发送者地址 from、接收者地址 to 和交易金额 amount 作为参数，然后根据这些参数创建一个新的交易

```

func (bc *Blockchain) CreateTransaction(from, to []byte, amount int)
(*transaction.Transaction, bool) {
    var inputs []transaction.TxInput
    var outputs []transaction.TxOutput
    // 定义了两个空的切片，inputs 用于存储输入，outputs 用于存储输出
    acc, validOutputs := bc.FindSpendableOutputs(from, amount)
    // 使用 FindSpendableOutputs 函数查找发送者的地址的可花费输出
    if acc < amount {
        fmt.Println("Not enough coins!")
        return &transaction.Transaction{}, false
    }
    // 如果找到的 UTXO 数量不够 amount，则打印“not enough coins!”并返回错误
    for txid, outidx := range validOutputs {
        txID, err := hex.DecodeString(txid)
        utils.Handle(err)
    }
}

```

```

        input := transaction.TxInput{txID, outidx, from}
        inputs = append(inputs, input)
    }
    //将找到的 UTXO 转换为输入，并将其添加到 inputs 切片中
    outputs = append(outputs, transaction.TxOutput{amount, to})
    if acc > amount {
        outputs = append(outputs, transaction.TxOutput{acc - amount, from})
    }
    //创建两个输出，一个是给接收者的amount，另一个是转出者的funds，并将它们添加到 outputs 切片中
    tx := transaction.Transaction{nil, inputs, outputs}
    tx.SetID()
    //创建了一个新的交易对象，并使用 SetID 函数设置交易 ID
    return &tx, true
}

func (bc *Blockchain) Mine(txs []*transaction.Transaction) {
    bc.AddBlock(txs)
}

```

main.go

###

```

package main

import (
    "GOBLOCKCHAIN/blockchain"
    "GOBLOCKCHAIN/transaction"
    "fmt"
)

func main() {
    // 创建一个空的交易池
    txPool := make([]*transaction.Transaction, 0)
    // 声明临时交易和其他变量
    var tempTx *transaction.Transaction
    var ok bool
    var property int

    // 创建一个区块链
    chain := blockchain.CreateBlockchain()
    // 查询 "Always" 的余额
    property, _ = chain.FindUTXOs([]byte("Always"))
    fmt.Println("Balance of Always: ", property)

    // 创建一笔交易，从 "Always" 转账给 "KFC"
    tempTx, ok = chain.CreateTransaction([]byte("Always"), []byte("ManBa"), 50)
    if ok {
        // 如果交易创建成功，将其添加到交易池中
        txPool = append(txPool, tempTx)
    }
    // 将交易池中的交易打包到新的区块中，并添加到区块链中
    chain.Mine(txPool)
    // 清空交易池
}

```

```

txPool = make([]*transaction.Transaction, 0)
// 再次查询 "Leo Cao" 的余额
property, _ = chain.FindUTXOs([]byte("Always"))
fmt.Println("Balance of Always: ", property)

// 创建一笔无效的交易, 从 "KFC" 转账给 "MDL"
tempTx, ok = chain.CreateTransaction([]byte("ManBa"), []byte("KFC"), 100) //
交易金额不足 交易失败
if ok {
    // 如果交易创建成功, 将其添加到交易池中
    txPool = append(txPool, tempTx)
}

// 创建两笔有效的交易
tempTx, ok = chain.CreateTransaction([]byte("Always"), []byte("ManBa"), 200)
if ok {
    txPool = append(txPool, tempTx)
}

tempTx, ok = chain.CreateTransaction([]byte("ManBa"), []byte("KFC"), 100)
if ok {
    txPool = append(txPool, tempTx)
}

// 将交易池中的交易打包到新的区块中, 并添加到区块链中
chain.Mine(txPool)
// 清空交易池
txPool = make([]*transaction.Transaction, 0)
// 再次查询 "Always" 的余额
property, _ = chain.FindUTXOs([]byte("Always"))
fmt.Println("Balance of Always: ", property)
// 查询其他用户的余额
property, _ = chain.FindUTXOs([]byte("ManBa"))
fmt.Println("Balance of ManBa: ", property)
property, _ = chain.FindUTXOs([]byte("KFC"))
fmt.Println("Balance of KFC: ", property)

// 遍历区块链中的每个区块, 并打印信息
for _, block := range chain.Blocks {
    fmt.Printf("Timestamp: %d\n", block.Timestamp)
    fmt.Printf("hash: %x\n", block.Hash)
    fmt.Printf("Previous hash: %x\n", block.PrevHash)
    fmt.Printf("nonce: %d\n", block.Nonce)
    fmt.Println("Proof of work validation:", block.ValidatePow())
}

// 尝试创建两笔额度不足的交易
tempTx, ok = chain.CreateTransaction([]byte("ManBa"), []byte("Always"), 300)
if ok {
    txPool = append(txPool, tempTx)
}

tempTx, ok = chain.CreateTransaction([]byte("KFC"), []byte("Always"), 100)
if ok {
    txPool = append(txPool, tempTx)
}

```

```

// 将交易池中的交易打包到新的区块中，并添加到区块链中
chain.Mine(txPool)
// 清空交易池
txPool = make([]*transaction.Transaction, 0)

// 再次遍历区块链中的每个区块，并打印信息
for _, block := range chain.Blocks {
    fmt.Printf("Timestamp: %d\n", block.Timestamp)
    fmt.Printf("hash: %x\n", block.Hash)
    fmt.Printf("Previous hash: %x\n", block.PrevHash)
    fmt.Printf("nonce: %d\n", block.Nonce)
    fmt.Println("Proof of work validation:", block.ValidatePow())
}

// 再次查询用户的余额
property, _ = chain.FindUTXOs([]byte("Always"))
fmt.Println("Balance of Always: ", property)
// 查询其他用户的余额
property, _ = chain.FindUTXOs([]byte("ManBa"))
fmt.Println("Balance of ManBa: ", property)
property, _ = chain.FindUTXOs([]byte("KFC"))
fmt.Println("Balance of KFC: ", property)
}

```

